Sisteme de operare

Laborator 10

Semestrul I 2023-2024

Vlad Olaru

Laborator 10

- advanced I/O
- programare client-server TCP/IP
  - servere concurente TCP
  - servere concurente UDP
  - servere multiprotocol

Multiplexarea I/O

- Q: cum citim date din doi descriptori de fisiere diferiti cand nu stim cand vor fi disponibile datele?
  - obs: citirea blocanta cu *read* nu functioneaza
- solutii posibile
  - *polling* (setam operatii neblocante pe descriptorii de fisiere si citim in bucla pe rand)
    - dezavantaje: proasta utilizare a CPU (putem folosi *sleep* pentru a ameliora situatia, dar nu stim nici cat sa asteptam !)
  - *I/O asincron*
    - instruim kernelul sa ne trimita un semnal (SIGPOLL/SIGIO) cand datele sunt disponibile
    - dezavantaj: exista un singur asemenea semnal per proces => la livrarea semnalului nu stim pe care descriptor sa citim (trebuie sa facem descriptorii non-blocanti si sa incercam citirea in secventa)
  - *multiplexarea I/O*
    - definim o lista de descriptori de fisiere de care suntem interesati
    - apelam o functie care se deblocheaza doar cand exista descriptori disponibili pt operatii de I/O (exista date disponibile, de ex)
    - dupa deblocare, exista functii care ne ajuta sa identificam acesti descriptori
    - ex: *select*, *poll*

select

- permite sa notificam kernelul in privinta
  - descriptorilor de interes
  - conditiilor in care ne intereseaza: read/write/exceptii
  - timpului de asteptare a disponibiliatii de I/O (deloc, pentru totdeauna sau pt o perioada specificata de timp)
- kernelul raspunde cu urmatoarele informatii
  - nr total de descriptori disponibili pt operatii de I/O
  - care descriptori sunt disponibili pentru fiecare tip de conditie: read/write/exceptie

```
int select(int nfds, fd_set *readfds, fd_set *writefds,
                     fd_set *exceptfds, struct timval *timeout);
```

Valori timeout

- NULL => se asteapta la infinit
  - *select* se deblocheaza cand un descriptor e gata de I/O sau la primirea unui semnal
  - la primirea unui semnal *select* intoarce -1 si errno = EINTR
- timeout.tv_sec == 0 && timeout.tv_usec == 0
  - *select* se intoarce imediat, fara blocare
  - se foloseste in polling
- timeout.tv_sec != 0 || timeout.tv_usec != 0
  - asteapta durata de timp specificata in secunde si microsecunde
  - *select* se deblocheaza fie cand un descriptor a devenit disponibil fie cand a expirat perioada de timp (fie la primirea unui semnal, ca mai sus)
  - daca timeout-ul a expirat si niciun descriptor n-a devenit disponibil, *select* intoarce valoarea 0

Seturi de descriptori

- *readfds*, *writefds*, *exceptfds* specifica descriptorii de interes pt operatiile de I/O corespunzatoare
- *fd_set* este un tip de date de tip bitmap, 1 bit per descriptor de interes
- operatii cu seturi de descriptori
  - alocarea unei variabile
  - asignarea de variabile
  - una dintre urmatoarele operatii pe bitmap-uri

```
FD_ZERO(fd_set *fds);          // pune bitii din fds pe 0
FD_SET(int fd, fd_set *fds);  // pune pe 1 bitul fd din fds
FD_CLR(int fd, fd_set *fds);  // pune pe 0 bitul fd din fds
FD_ISSET(int fd, fd_set *fds);  // testeaza daca bitul fd din fds e setat
```

Seturi de descriptori (cont.)

- secventa tipica de operare

```
fd_set rfds;
int fd;
…
FD_ZERO(&rfds);
FD_SET(fd, &rfds);
if(select(nfds, &rfds, ….) < 0)
        exit(1);
if(FD_ISSET(fd, &rfds))
        read(fd, buf, sizeof(buf) – 1); // read neblocant !
```

Nr max de descriptori si valoare de retur

- cand nu stim valorile descriptorilor primul parametru a lui *select* se alege ca fiind nr maxim de file descriptori posibil
  - FD_SETSIZE
- daca stim valoarea maxima a descriptorilor pe care ii folosim, atunci o folosim ca valoare a primului argument de apel al lui *select*
  - FD_SETSIZE poate fi mare (1024) => optimizare a cautarilor kernelului pt descriptori disponibili
- *select* intoarce:
  - -1 in caz de eroare
  - 0 pt timeout expirat si niciun descriptor disponibil
  - > 0 : nr de descriptori disponibili pt operatii de I/O
    - cei pt care operatiile de R/W nu se vor bloca
    - respectiv, descriptorii pt care exista o conditie de exceptie in asteptare (eg, OOB data in conexiuni de retea)

Operatii scatter/gather

- permit citirea/scrierea din/in buffere necontigue intr-un singur apel
- scatter read/ gather write
- folosesc vectori de buffere

```
struct iovec {
        void *iov_base;    // adresa de inceput a bufferului
        size_t iov_len;    // dimensiunea bufferului
}
ssize_t readv(int fd, const struct iovec iov[], int iovcnt);
ssize_t writev(int fd, const struct iovec iov[], int iovcnt);
```

Operatii scatter/gather (cont.)

ssize_t readv(int fd, const struct iovec iov[], int iovcnt);

- imprastie datele in buffere in ordine
- intotdeauna umple un buffer inainte sa treaca la urmatorul
- intoarce nr total de octeti cititi sau 0 daca nu mai sunt date (EOF)

ssize_t writev(int fd, const struct iovec iov[], int iovcnt);

- aduna datele de output din buffere in ordine *iov[0], iov[1], … iov[iovcnt -1]*
- intoarce nr total de octeti scrisi (in mod normal, suma dimensiunii bufferelor)